

APIs Java: Enum e Tipos genéricos

CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Enum

O tipo Enum

- O tipo Enum é usado para **definir um conjunto fixo de constantes**
 - Exemplo: estações do ano, dias da semana, meses do ano, planetas do sistema solar
- Antes da introdução do Enum no JDK1.5, um padrão de projeto era declarar um grupo de constantes inteiras

```
public class Teste{  
    public static final int DOMINGO = 0;  
    public static final int SEGUNDA = 1;  
    public static final int TERCA   = 2;  
    // .....  
    public void aulas(){  
        int aulasPoo[] = {SEGUNDA, TERCA};  
    }  
}
```

Enum para dias da semana

Definição

Classes que exportam uma única instância para cada constante enumerada via um atributo público, estático e final.

```
// Arquivo DiasDaSemana.java
public enum DiasDaSemana{
    DOMINGO, SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO
}
```

```
// Arquivo Teste.java
public class Teste{
    public void aulas(){
        DiasDaSemana aulasPoo[] = {DiasDaSemana.SEGUNDA, DiasDaSemana.TERCA};
    }
}
```

Enum para cores da Shell

```
public enum Cor{
    PRETO(0), VERMELHO(1), VERDE(2), AMARELO(3),
    AZUL(4), ROXO(5), CIANO(6), BRANCO(7);

    public final int codigo;

    Cor(int c){
        this.codigo = c;
    };

    public static Cor getByCodigo(int c){
        for (Cor cor: Cor.values()){
            if (c == cor.codigo){
                return cor;
            }
        }
        throw new IllegalArgumentException("código inválido");
    }
}
```

Enum para cores da Shell

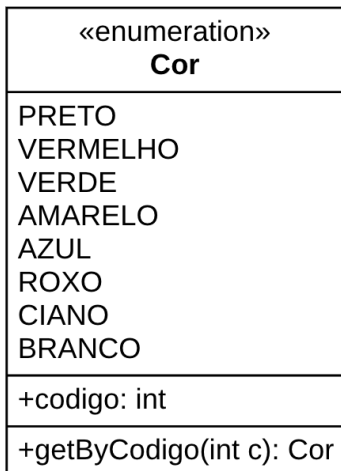
```
public class Teste{

    public static void main(String[] args) {
        Cor corDoFundo = Cor.getByCodigo(0);
        Cor corDoTexto = Cor.getByCodigo(7);

        System.out.println("Cor do fundo: " + corDoFundo);

        switch(corDoFundo){
            case PRETO:
                switch(corDoTexto){
                    case PRETO:
                        System.out.println("não me parece uma boa combinação");
                        break;
                    case BRANCO:
                        System.out.println("me parece uma boa combinação");
                        break;
                }
            }
        }
    }
}
```

Representação de Enum em UML



Percorrer todos os valores de uma Enumeração

```
public class App{  
  
    public static void main(String[] args) {  
  
        for (Cor cor: Cor.values()){  
            System.out.println(cor);  
        }  
  
    }  
}
```

Exercício 1: Sistema solar

- Fazer um Enum para representar os planetas do sistema solar
- Para cada planeta é importante guardar qual a sua posição em relação ao sol (e.g. A Terra está na posição 3)

Exercício 2: Baralho francês

- Desenvolva um conjunto de classes que possibilite representar um baralho de cartas
- Um baralho francês possui 52 cartas, sendo 13 cartas para cada um dos 4 naipes (paus, ouros, copas e espadas)
- Cartas são: Ás, 2, 3, 4, 5, 6, 7, 8, 9, 10, Valete (J), Dama (Q) e Rei (K)



Tipos genéricos em Java

Classe capaz de armazenar instâncias de qualquer classe

```
public class Caixa{  
    private Object dado;  
  
    public void set(Object obj){  
        this.dado = obj;  
    }  
  
    public Object getDado(){ return this.dado;}  
}
```

```
public class Principal{  
    public static void main(String[] args){  
        Caixa c = new Caixa();  
        String s = "Olá mundo";  
        c.set(s);  
        // typecasting obrigatório  
        String outra = (String) c.getDado();  
    }  
}
```

Classe capaz de armazenar instâncias de qualquer classe

```
Caixa c = new Caixa();
Caixa d = new Caixa();
String s = "Olá mundo";
Pessoa p = new Pessoa("Joao");
c.set(s); // OK
d.set(p); // OK
String nova = (String) c.getDado();
// só apresentará erro durante a execução
String outra = (String) d.getDado(); // ERRO durante execução!!
```

- É mais fácil detectar *bugs* em tempo de compilação do que em tempo de execução.

Classe capaz de armazenar instâncias de qualquer classe

```
Caixa c = new Caixa();  
Caixa d = new Caixa();  
String s = "Olá mundo";  
Pessoa p = new Pessoa("Joao");  
c.set(s); // OK  
d.set(p); // OK  
String nova = (String) c.getDado();  
// só apresentará erro durante a execução  
String outra = (String) d.getDado(); // ERRO durante execução!!
```

- É mais fácil detectar *bugs* em **tempo de compilação** do que em **tempo de execução**.

Tipos genéricos

Possibilitam que bugs possam ser detectados já na fase de compilação

Tipos genéricos em Java

- Permite que tipos (Classes ou Interfaces) sejam passados como parâmetros durante a definição de Classes, Interfaces ou métodos
- **Parâmetros de tipos permite reutilizar a mesma classe** diante de diferentes entradas
- Entradas de parâmetros formais são obrigatoriamente valores

```
public void metodo(int parametro){ .... }  
....  
objeto.metodo(123); // parâmetro deve ser um valor
```

- **Entradas de parâmetros de tipos** são tipos
 - Exemplo de tipos: String, Pessoa, Carro, etc.

Reescrevendo classe Caixa para usar tipos genéricos

```
public class Caixa<T>{  
    private T dado;  
  
    public void set(T obj){  
        this.dado = obj;  
    }  
  
    public T getDado(){ return this.dado;}  
}
```

```
public class Principal{  
    public static void main(String[] args){  
        Caixa<String> c = new Caixa<>();  
        String s = "Olá mundo";  
        c.set(s);  
        String outra = c.getDado(); // não precisa do typecasting  
    }  
}
```

Reescrevendo classe Caixa para usar tipos genéricos

```
public class Caixa<T>{  
    private T dado;  
  
    public void set(T obj){  
        this.dado = obj;  
    }  
  
    public T getDado(){ return this.dado;}  
}
```

```
Caixa<String> c = new Caixa<>();  
  
Pessoa p = new Pessoa("Joao");  
  
c.set(p); // erro de compilação, tipos errados
```

Parâmetro de tipo limitado

- É possível restringir quais tipos são aceitos
 - Somente as classes ou interfaces da hierarquia seriam permitidas

```
public class Caixa<T extends Personagem>{  
    private T personagem;  
    .....  
}
```

```
public class Principal{  
    public static void main(String[] args){  
  
        // ok, pois Aldeao herda de Personagem  
        Caixa<Aldeao> c = new Caixa<>();  
  
        Caixa<String> n = new Caixa<>();// erro!  
    }  
}
```

Convenção de nomes

- Nomes dos parâmetros de tipos devem ser uma única letra maiúscula
 - E – para elementos
 - T – para tipos
 - K – para chaves
 - V – para valores
 - N – para números
- A API Collections do Java faz uso do nome *E*
- Veja mais informações na documentação oficial
<https://docs.oracle.com/javase/tutorial/java/generics>